Complex and Social Networks

Lab 1
Reproducing graphs

Autumn 2026

Code:

Hana Dubovska (hana.dubovska@estudiantat.upc.edu)
Ida Kallevik Tollaksen
(ida.kallevik.tollaksen@estudiantat.upc.edu)

UPC

September, 2026

Contents

1 Introduction **2**

2 Process **2**

2.1 Graph A 2

2.2 Graph B 3

3 Results **4**

3.1 Graph A 4

3.2 Graph B 4

4 Conclusion **6**

5 Declarations **6**

5.1 Artificial intelligence 6

1 Introduction

The goal of this lab is to reproduce A and B. As a group we decided to split the work and replicate one graph each. Hana Dubovska completed graph A and Ida Kallevik Tollaksen completed graph B.

Graph A is a Watts-Strogatz model showing how clustering and the shortest path is affected as p increases.

Graph B is an Erdős–Rényi model showing the shortest path length as the size of a network(parameter n) increases.

2 Process

2.1 Graph A

The proposition and idea behind this is that even small amount of randomness can reduce the distance between nodes even while preserving local clustering.

Already from looking at the graph we can draw several conclusions which are important for reproduction - like p values range, normalization process and that the x axis is logarithmic.

In the code at first I define p values in a vector, these range from 0.0001 to 1. Smaller p means that the network is regular and increasing p means that more connections are randomly rewired.

I also created two empty vectors C - for coefficient values and L - for average path lengths.

Then I build a network for each of those p values that I defined using a loop. Dimension is set to 1 so the nodes are arranged in a one dimensional space, there are 1000 nodes in the network and each is connected to 5 nearby nodes.

Afterwards I calculate clustering coefficient using a transitivity function and saving the values to C vector. And I calculate average shortest path using a mean distance function and save the result in the L vector.

Another important step is to create a reference network with p set to 0. In this one there is no rewiring so it can be used as a baseline for comparison. And using these values we can normalize the results as is stated in the assignment.

Lastly I plot the vectors with clustering coefficient values and average path lengths, x axis was set to be logarithmic because p values range from very small values and it

would be difficult to see without this transformation.

2.2 Graph B

The goal of graph B is to show how the average shortest-path length of an Erdős–Rényi graph changes as the network size increases.

The first step was picking a range of n values(network size) to test. I started with n = 10 as the smallest value for every run, and then tried out a few different upper limits before settling on 8,000, 10,000, 15,000 and 20,000 as the biggest. I ended on these values based on what my laptop could handle. The runtime was quite long once n reached 20,000, so because of this I used that as an upper limit.

I set the connectivity constant (c) to 1.5. This value needs to be more than 1, since the lab shows that if p > (1 + ϵ)ln n/n, the graph will almost certainly be connected. By using 1.5 we get a good margin for the connectivity threshold.

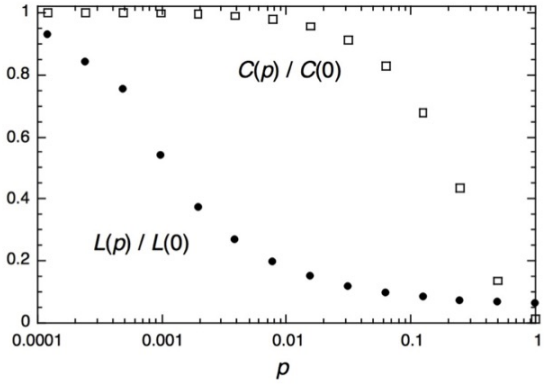
For each n value I calculated p using the formula p = c · ln(n) / n from the lab material. This helps make sure p scales correctly with n and that the graph's connectivity does not disconnect.

For every n value, I generated 10 separate ER graphs (n_reps <- 10) using sample_gnp(n, p), and then calculated the average shortest-path length of each one using mean_distance(). Then I averaged these 10 results into a single L value per n.

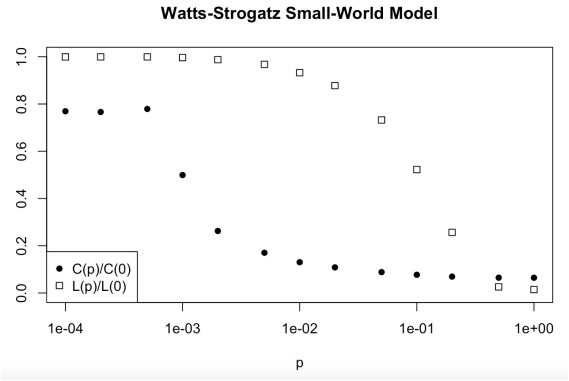
Finally, I plotted these average shortest-path values (L_mean) against n, to show how the shortest-path length grows as the network size increases.

3 Results

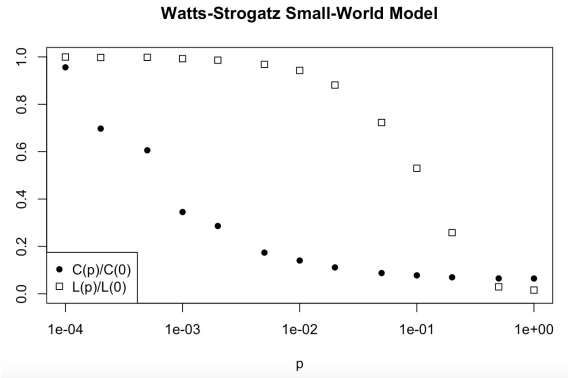
3.1 Graph A



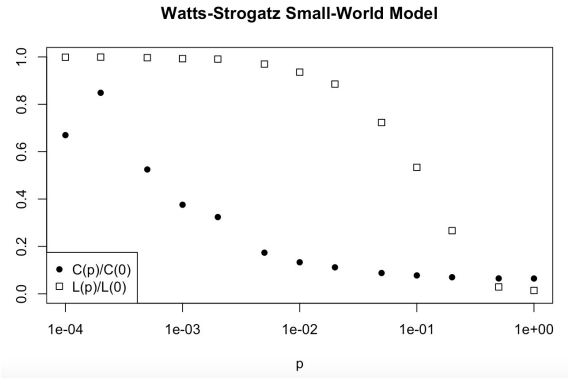
(a) Original



(b) Run 1



(c) Run 2



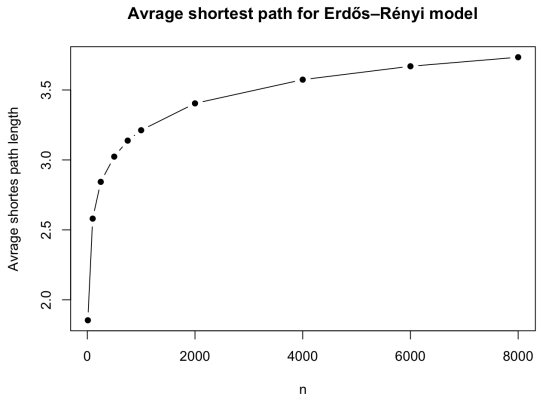
(d) Run 3

Figure 1: Graph A results for 3 different runs

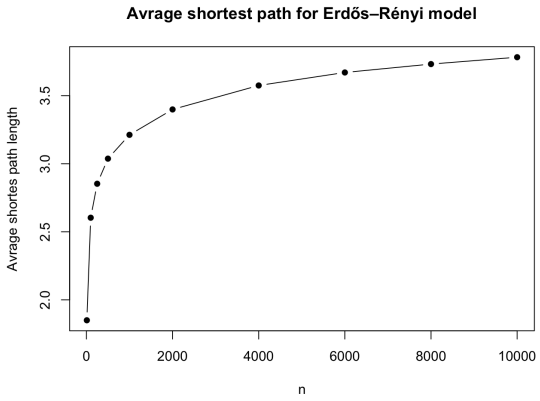
As experiment, since I could see that the graph I generated was slightly different I tried to run the code 3 times. In each run the points for lowest p values were moving around a bit. This is because the model is random and when the p is very small, only a small number of edges are rewired and therefore the change is more noticeable.

3.2 Graph B

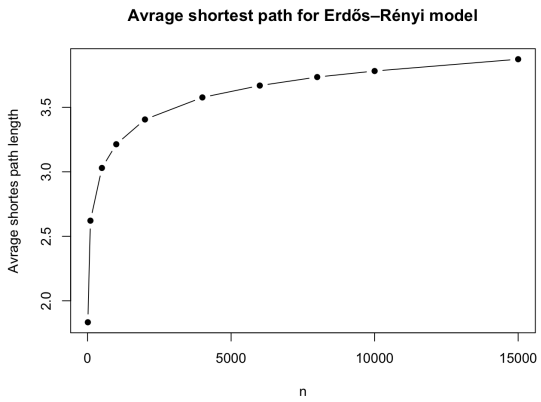
The results for graph B were four runs, each using n values from 10 up to my different upper limits. The runtime for the run with n = 20,000 was roughly 3 minutes.



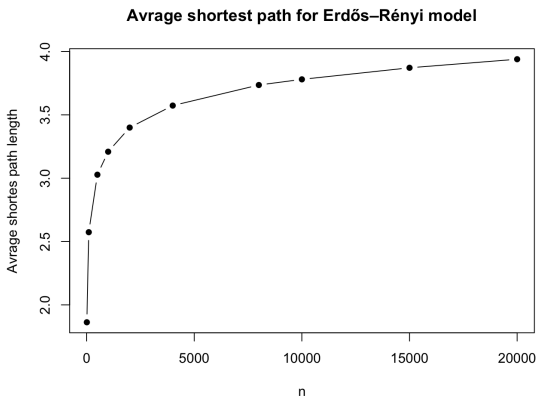
(a) N = 10 - 8,000



(b) N = 10 - 10,000



(c) N = 10 - 15,000



(d) N = 10 - 20,000

Figure 2: Graph B results with different N values

Over all 4 runs we can see a similar graph shape. The average shortest-path length increases a lot for smaller values of n but evens out more at around 3.5 for larger n values. This is expected for an Erdős–Rényi model, because as more nodes are added, the network grows and gets more connections. Meaning nodes stay reachable through short paths instead of the distance continuing to grow at the same rate.

Comparing the 4 result graphs, we can see that increasing the upper limit of n mostly just extends the curve. This suggests the trend is consistent across different ranges, not just the values I chose.

4 Conclusion

All graphs from our results matched the expected shapes from the lab material. Graph A showed how a small amount of rewiring can drastically shorten path length while keeping clustering high. Graph B showed the average shortest-path length growing quickly at first before flattening out as n increased. It's worth noting though that for graph B we limited n to 20,000 instead of the original 200,000 because of runtime. The growth pattern still matched the expected outcome.

5 Declarations

5.1 Artificial intelligence

For part concentrating on graph A, AI (Claude and Chat GPT) was used as a support during coding process (debugging), and help when generating the figures to be able to replicate the design of the original one. Then also for slight grammatical corrections during report writing.

I, Ida Kallevik Tollaksen, worked on graph B in this lab. I had never worked with the language R or RStudio before, and therefore used AI (Claude Sonnet 5) for assistance in setting everything up and generally familiarizing myself with RStudio. It was also used to explain the material of the lab. For graph B, I wrote all the code myself. AI was used to review my code, point out bugs, and explain concepts, but did not generate any code. It was also used for general spellcheck.